# Structure and Union

Structures- array of structures - nested structures - structures with functions - self-referential structures - bit fields - pointers to structures, Unions- comparison with structures.

# Structure

- A structure is a user-defined data type that can be used to group items of possibly different types into a single type. The struct keyword is used to define a structure.
- Syntax of Structure
  - Structure Definition
  - Creating Structure Variables

## Structure Definition

```
struct structure_name {
    data_type1 member1;
    data_type2 member2;
    …
};
```

## Creating Structure Variable

```
struct strcuture_name var;
```

Or

```
struct structure_name {
    …
}var1, var2….;
```

```
struct Person
{
char name[50];
int id;
float salary;
};

int main()
{  struct Person p1;
return 0;
}
```

```
struct Person
{
char name[50];
int id;
float salary;
}p1;
```

# Keyword typedef

- We use the typedef keyword to create an alias name for data types. It is commonly used with structures to simplify the syntax of declaring variables.

```
// struct with typedef person
typedef struct Person {
char name[50];
int id;
float salary;
} person;

// create  Person variable
person p1;
```

# Basic Operations of Structure

- Access Structure Members
  - structure_name . member1;
  - strcuture_name . member2;

- If we have a pointer to the structure, we can also use the arrow operator to access the members.
  - structure_ptr -> member1
  - structure_ptr -> member2

# Initialize Structure Members

- struct structure_name str;
- str.member1 = value1;
- ….

- struct structure_name str = {value1, value2, value3 ….};

- struct structure_name str = { .member1 = value1, .member2 = value2, .member3 = value3 };

# Array within a Structure

- An array can be declared inside a structure as a member when we need to store multiple members of the same type.
- Syntax:

```
struct StructureName {
 // Other members
dataType arrayName[arraySize];
};
```

- Accessing Elements of an Array within a Structure

```
structureName.arrayName[index]
```

```
struct student{
        char name[20];
        int roll;
        float marks[5]; /* This is array within structure */
};
int main(){
        struct student s;
        int i;
        float sum=0, p;
        printf("Enter name and roll number of students:\n");
        scanf("%s%d",s.name, &s.roll);
        printf("\nEnter marks obtained in five different subject\n");
        for(i=0;i< 5;i++) {
                printf("Enter marks:\n");
                scanf("%f",&s.marks[i]);
                sum = sum + s.marks[i];
        }
        p = sum/5;
        printf(“Student records and percentage is:\n”);
        printf("Name : %s\n", s.name);
        printf("Roll Number : %d\n", s.roll);
        printf("Percentage obtained is: %f", p);
        return 0;
}
```

# Array of Structures

- to store several structures of the same type in a single array.

```
struct student
{
    char stu_name[30];
    float cgpa;
    int age;
    char reg_no[10];
};
struct student s[70];
```

```c
#include <stdio.h>
#include <string.h>
// Structure definition
struct Student {
    char name[50];
    int age;
    float marks;
};
int main() {
     // Declaration and initialization of an array of structures
    struct Student students[3] = {
        {"Nikhil", 20, 85.5},
        {"Shubham", 22, 90.0},
        {"Vivek", 25, 78.0}
    };
```

```c
 // Traversing through the array of structures
//and displaying the data
    for (int i = 0; i < 3; i++) {
        printf("Student %d:\n", i+1);
        printf("Name: %s\n", students[i].name);
        printf("Age: %d\n", students[i].age);
        printf("Marks: %.2f\n", students[i].marks);
    }
    return 0;
}
```

```
#include <stdio.h>
#include <string.h>
// Structure definition
struct Student {
   char name[50];
   int age;
   float marks;
};
int main() {
     // Declaration and initialization of an array of structures
   struct Student students[70];
   for (int i=0;i<70;i++){
      scanf(“%s”, students[i].name);
     scanf(“%d”, &students[i].age);
     scanf(“%f”, &students[i].marks);
}
```

```
 // Traversing through the array of structures
//and displaying the data
   for (int i = 0; i < 70; i++) {
      printf("Student %d:\n", i+1);
      printf("Name: %s\n", students[i].name);
      printf("Age: %d\n", students[i].age);
      printf("Marks: %.2f\n", students[i].marks);
   }
   return 0;
}
```

# Structure within Structures

- Nested Structure
- One of the elements in the definition of a struct type is of another struct type.
- Syntax

```
struct struct1{
  type var1;
  type var2;
  struct struct2 strvar;
}
```

# Example

```
#include <stdio.h>
#include <string.h>
struct dob{
   int d, m, y;
};
struct employee{
   char name[10];
   float salary;
   struct dob d1;
};
```

```
int main()
{
   struct employee e1 = {"Kiran", 25000, {12, 5, 1990}};
   printf("Name: %s\n", e1.name);
   printf("Salary: %f\n", e1.salary);
   printf("DoB: %d-%d-%d\n", e1.d1.d, e1.d1.m, e1.d1.y);
    return 0;
}
```

```
struct dob d1 = {12, 5, 1990};
struct employee e1 = {"Kiran", 25000, d1};
```

```
struct employee{
   char name[10];
   float salary;
   struct dob{
      int d, m, y;
   } d1;
};
```

# Structure Pointer - **Pointers to Structures**

- A structure pointer is a pointer variable that stores the address of a structure.
- Define a new derived data type using the "struct" keyword.
- Declare a structure variable of this derived data type.
- Declare a pointer variable and store the address of previously declared structure variable.
- To access the elements of a structure with pointer, we use a special operator called the indirection operator ->

# Structure Pointer - **Pointers to Structures**

```
#include <stdio.h>
struct Student {
    int id;
    char name[20];
};
int main() {
    struct Student s;
    struct Student *ptr;
    ptr = &s;   // pointer points to structure
    printf("Enter student ID: ");
    scanf("%d", &ptr->id);
    printf("Enter student name: ");
    scanf("%s", ptr->name);
    printf("\nStudent Details:\n");
    printf("ID: %d\n", ptr->id);  // printf("ID: %d\n", (*ptr).id);
    printf("Name: %s\n", ptr->name); //printf("Name: %s\n", (*ptr).name);
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
struct Student
{
   int roll;
   char name[30];
   float marks;
};
int main()
{
   struct Student *s;
   int n, i;
   printf("Enter number of students: ");
   scanf("%d", &n);
   /* Dynamic memory allocation */
   s = (struct Student *)malloc(n * sizeof(struct Student));
   if (s == NULL)
   {
      printf("Memory allocation failed\n");
      return 1;
   }
   /* Reading student details */
   for (i = 0; i < n; i++)
   {
      printf("Roll number: ");
      scanf("%d", &s[i].roll);
      printf("Name: ");
      scanf("%s", s[i].name);
      printf("Marks: ");
      scanf("%f", &s[i].marks);
   }
   /* Displaying student details */
   printf("\n--- Student Details ---\n");
   for (i = 0; i < n; i++)
   {
      printf("Roll: %d\n", s[i].roll);
      printf("Name: %s\n", s[i].name);
      printf("Marks: %.2f\n", s[i].marks);
   }

   /* Free allocated memory */
   free(s);

   return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
struct Student
{
   int roll;
   char name[30];
   float marks;
};
int main()
{
   struct Student *s;
   int n, i;
   printf("Enter number of students: ");
   scanf("%d", &n);
   /* Dynamic memory allocation */
   s = (struct Student *)malloc(n * sizeof(struct Student));
   if (s == NULL)
   {
      printf("Memory allocation failed\n");
      return 1;
   }
for (i = 0; i < n; i++)
   {
    /* Reading student details */
      printf("Roll number: ");
      scanf("%d", &(s + i)->roll);
      printf("Name: ");
      scanf("%s", (s + i)->name);
      printf("Marks: ");
      scanf("%f", &(s + i)->marks);
   }
 /* Displaying student details */
   for (i = 0; i < n; i++)
   {
      printf("Roll: %d\n", (s + i)->roll);
      printf("Name: %s\n", (s + i)->name);
      printf("Marks: %.2f\n", (s + i)->marks);
   }
   /* Free allocated memory */
   free(s);

   return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
struct Student
{
   int roll;
   char name[30];
   float marks;
};
int main()
{
   struct Student *s,*temp;
   int n, i;
   printf("Enter number of students: ");
   scanf("%d", &n);
   /* Dynamic memory allocation */
   s = (struct Student *)malloc(n * sizeof(struct Student));
   if (s == NULL)
   {
      printf("Memory allocation failed\n");
      return 1;
   }
   temp=s;
   for (i = 0; i < n; i++) /* Reading student details */
      {
         printf("Roll number: ");
         scanf("%d", &(s)->roll);
         printf("Name: ");
         scanf("%s", (s )->name);
         printf("Marks: ");
         scanf("%f", &(s)->marks);
         s++;
      }
      s=temp;
   for (i = 0; i < n; i++) /* Displaying student details */
      {
         printf("Roll: %d\n", (s)->roll);
         printf("Name: %s\n", (s)->name);
         printf("Marks: %.2f\n", (s)->marks);
         s++;
      }
      /* Free allocated memory */
      free(temp);
      temp=NULL;
      return 0;
   }
```

# Structures and Functions

# How to Pass Struct Elements

```
#include <stdio.h>
struct rectangle{
  float len, brd;
};
void area(float a, float b){
  double ar = (double)(a*b);
  printf("Length: %f \nBreadth: %f \nArea: %lf\n", a, b, ar);
}
int main(){
  struct rectangle r;
  printf("Enter length and breadth\n");
  scanf("%f%f",&r.len,&r.brd);
  area(r.len, r.brd);
  return 0;
}
```

# How to Pass Struct Variable

```
#include <stdio.h>
struct rectangle{
  float len, brd;
  double a;
};
void area(struct rectangle r1){
  r1.a = (double)(r1.len*r1.brd);
  printf("Length: %f \nBreadth: %f \nArea: %lf\n", r1.len, r1.brd, r1.a);
}
int main(){
  struct rectangle r;
  printf("Enter length and breadth\n");
  scanf("%f%f",&r.len,&r.brd);
  area(r);
  return 0;
}
```

# How to return Struct Variable

```
#include <stdio.h>
struct rectangle {
  float len, brd;
  double a;
};
struct rectangle area(float x, float y){
  double ar = (double)(x*y);
  struct rectangle r1 = {x, y, ar};
    return r1;
}
int main(){
  struct rectangle r;
  float x, y;
  printf("Enter length and breadth\n");
  scanf("%f%f",&x,&y);
  r = area(x, y);
  printf("Length: %f \n Breadth: %f \n Area: %lf\n", r.len, r.brd, r.a);
  return 0;
}
```

# How to Pass Struct Variable by a reference

```
#include <stdio.h>
struct rectangle{
  float len, brd;
  double a;
};
void area(struct rectangle *r1){
  r1 -> a = (double)(r1 -> len * r1 -> brd);
  printf("Length: %f \nBreadth: %f \nArea: %lf\n", r1->len, r1->brd, r1->a);
}
int main(){
  struct rectangle r;
  printf("Enter length and breadth\n");
  scanf("%f%f",&r.len,&r.brd);
  area(&r);
  return 0;
}
```

# How to Pass Struct pointer

```
#include <stdio.h>
struct rectangle{
  float len, brd;
  double a;
};
void area(struct rectangle *r1){
  r1 -> a = (double)(r1 -> len * r1 -> brd);
  printf("Length: %f \nBreadth: %f \nArea: %lf\n", r1->len, r1->brd, r1->a);
}
int main(){
  struct rectangle r, *rp;
  rp = &r;
  printf("Enter length and breadth\n");
  scanf("%f%f",&r.len,&r.brd);
  area(rp);
  return 0;
}
```

# How to Pass Struct pointer

```
#include <stdio.h>
#include <stdlib.h>
struct rectangle{
  float len, brd;
  double a;
};
void area(struct rectangle *r1){
  r1 -> a = (double)(r1 -> len * r1 -> brd);
  printf("Length: %f \nBreadth: %f \nArea: %lf\n", r1->len, r1->brd, r1->a);
}
int main(){
  struct rectangle *rp;
  rp=(struct rectangle *)malloc(sizeof(struct rectangle));
  printf("Enter length and breadth\n");
  scanf("%f%f",&rp->len,&rp->brd);
  area(rp);
  return 0;
}
```

# How to return Struct pointer

- r1 is created on the stack
- It exists only while the function is executing
- When area() finishes, r1 is destroyed
- The returned pointer points to invalid memory

```c
#include <stdio.h>
struct rectangle {
  float len, brd;
  double a;
};
struct rectangle * area(float x, float y){
  double ar = (double)(x*y);
  static struct rectangle r1; // Static ensures the object remains in memory after function exits
  r1.len = x; r1.brd = y; r1.a = ar;
  return &r1;
}
int main (){
    struct rectangle *r;
  float x, y;
    printf("Enter length and breadth\n");
  scanf("%f%f",&x,&y);
  r = area(x, y);
  printf("Length: %f \n Breadth: %f \n Area: %lf\n", r->len, r->brd, r->a);
    return 0;
}
```

```
/tmp/mf0cojJlqY/main.c: In function 'area':
/tmp/mf0cojJlqY/main.c:10:11: warning: func
    local variable [-Wreturn-local-addr]
   10 |     return &r1;
      |            ^~~
Enter length and breadth
1
2
Segmentation fault


=== Code Exited With Errors ===
```

# Self-referential Structures

- A self-referential structure is a struct data type in C, where one or more of its elements are pointer to variables of its own type.
- They are extensively used to build complex and dynamic data structures such as linked lists and trees.

```
strut typename{
    type var1;
    type var2;
    … …
    struct typename *var3;
}
```

```c
#include <stdio.h>

struct Node {
    int data;
    struct Node *next;   // self-referential pointer
};

int main() {
    struct Node n1, n2;

    n1.data = 10;
    n1.next = &n2;   // n1 points to n2

    n2.data = 20;
    n2.next = NULL;  // last node

    printf("First node data: %d\n", n1.data);
    printf("Second node data: %d\n", n1.next->data);

    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
   int data;
   struct Node *next;   // self-referential pointer
};
int main() {
   struct Node *head, *second;
   // Allocate memory
   head = (struct Node *)malloc(sizeof(struct Node));
   second = (struct Node *)malloc(sizeof(struct Node));
   // Assign values
   head->data = 10;
   head->next = second;
   second->data = 20;
   second->next = NULL;
   // Access data
   printf("First node data  : %d\n", head->data);
   printf("Second node data : %d\n", head->next->data);
   // Free memory
   free(head);
   free(second);
   return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};
int main() {
   struct Node *head, *second, *third, *temp;
   // Allocate memory
   head   = (struct Node *)malloc(sizeof(struct Node));
   second = (struct Node *)malloc(sizeof(struct Node));
   third  = (struct Node *)malloc(sizeof(struct Node));

   // Assign data and links
   head->data = 10;
   head->next = second;
   second->data = 20;
   second->next = third;
   third->data = 30;
   third->next = NULL;

   // Traversal
   temp = head;
   printf("Linked List Elements:\n");

   while (temp != NULL) {
      printf("%d -> ", temp->data);
      temp = temp->next;
   }
   printf("NULL\n");

   // Free memory
   free(head);
   free(second);
   free(third);

   return 0;
}
```

# Bit Fields

- Bit fields allow us to store data in individual bits inside a structure instead of using full bytes or words.

```
struct structure_name {
   data_type member_name : number_of_bits;
};
```

```c
#include <stdio.h>
struct Normal {
    unsigned int a;
    unsigned int b;
    unsigned int c;
};
int main() {
    printf("Size of Normal structure: %lu bytes\n", sizeof(struct Normal));
    return 0;
}
```

```
Size of Normal structure: 12 bytes
```

```c
#include <stdio.h>
struct BitField {
    unsigned int a : 1;
    unsigned int b : 1;
    unsigned int c : 1;
};
int main() {
    printf("Size of BitField structure: %lu bytes\n", sizeof(struct BitField));
    return 0;
}
```

```
Size of BitField structure: 4 bytes
```

- **1. Memory optimization**
- Bit fields allow you to store **many boolean or small-range values** inside a single integer.
- **2. Hardware register mapping**
- Hardware registers often have **specific bits with specific meanings**.

# Union

- Union is a user-defined data type that can contain elements of the different data types just like structure.
- But unlike structures, all the members in the C union are stored in the same memory location.
- Due to this, only one member can store data at the given point in time.

```c
#include <stdio.h>

// Declaring multiple unions
union A{
    int x;
    char y;
};

union B{
    int arr[10];
    char y;
};

int main() {
    printf("Sizeof A: %ld\n", sizeof(union A));
    printf("Sizeof B: %ld\n", sizeof(union B));
    return 0;
}
```

```
Sizeof A: 4
Sizeof B: 40
```

```c
#include <stdio.h>

// Declaring multiple structures
struct SA{
    double x;
    char y;
};

struct SB{
    int arr[10];
    char y;
};

int main() {
    printf("Sizeof A: %ld\n", sizeof(struct SA));
    printf("Sizeof B: %ld\n", sizeof(struct SB));
    return 0;
}
```

Structures use padding to satisfy alignment requirements.
The structure size must be a multiple of the largest data type alignment inside it.

```
Sizeof A: 16
Sizeof B: 44
```

```c
#include <stdio.h>
union utest {
   int x, y;
};

int main(){
   union utest ut;
   printf("Address of union x is %p\n",&ut.x);
   printf("Address of union y is %p\n",&ut.y);
   ut.x = 2;
   printf("x = %d, y = %d\n", ut.x, ut.y);
   ut.y = 10;
   printf("x = %d, y = %d\n\n", ut.x, ut.y);
   return 0;
}
```

```c
#include <stdio.h>
struct stest {
   int x, y;
};

int main(){
   struct stest st;
   printf("Address of struct x is %p\n",&st.x);
   printf("Address of struct y is %p\n",&st.y);
   st.x = 2;
  printf("x = %d, y = %d\n", st.x, st.y);
   st.y = 10;
   printf("x = %d, y = %d", st.x, st.y);
   return 0;
}
```

```
Address of union x is 0x7ffc224f5e6c
Address of union y is 0x7ffc224f5e6c
x = 2, y = 2
x = 10, y = 10
```

```
Address of struct x is 0x7ffc224f5e70
Address of struct y is 0x7ffc224f5e74
x = 2, y = 32764
x = 2, y = 10
```

```c
#include <stdio.h>
union utest {
    int x;
    char y;
};

int main(){
    union utest ut;
    printf("Address of union x is %p\n",&ut.x);
    printf("Address of union y is %p\n",&ut.y);
    ut.x = 2;
    printf("x = %d, y = %d\n", ut.x, ut.y);
    ut.y = 'a';
    printf("x = %d, y = %d\n\n", ut.x, ut.y);
    return 0;
}
```

```
Address of union x is 0x7ffc766c383c
Address of union y is 0x7ffc766c383c
x = 2, y = 2
x = 97, y = 97
```

- 1. Memory-efficient data storage

When a variable can take one of many data types, but not all at the same time.

- 2. Implementing variant data types (tagged data)

Often combined with a struct to indicate what type of data is currently stored.